TRIBHUVAN UNIVERSITY

Faculty of Humanities and Social Sciences

Pashupati Multiple Campus

A Project Proposal on

HAMRO PRACHIN SAMPADA: A WEB BASED HERITAGE EXPLORER OF NEPAL

Submitted to

Department of Computer Application

Pashupati Multiple Campus

In partial fulfillment of the requirements for the degree of

Bachelor of Computer Applications (BCA)

Project III (CACS452), Eighth Semester

Submitted by

Niranjan Katwal

TU Symbol No.: 27802063

TU Registration No.: 6-2-278-18-2022

Project Supervisor

Name and designation of the supervisor

Month and year of submission

CHAPTER 1: INTRODUCTION

1.1 Background

Nepal has a dense concentration of living heritage: temples, stupas, palaces, courtyards and pilgrim sites that are still used for worship and festivals. The Kathmandu Valley alone is a UNESCO World Heritage property made up of seven monument zones [1]. Further sites are spread across all seven provinces. Information about them is scattered over travel sites, social media and printed guidebooks. It is rarely in one place or in Nepali and English together, and it is not organized so that a visitor can plan a day or a student can practice.

Games and interactive media are a recognized way to make heritage engaging. A survey of serious games in cultural heritage [2] describes how play and storytelling help visitors learn about places. Recommender systems [3], adaptive testing based on the Elo rating [4] and spaced repetition [5] are well-studied techniques that suit this setting: a visitor needs suggestions, a learner needs questions of the right difficulty, and a traveler with limited hours needs a sensible route [6].

This project proposes **Hamro Prachin Sampada** (हाम्रो प्राचीन सम्पदा, "our ancient heritage"), a free bilingual web application for exploring Nepal's heritage. The system holds a verified catalog of 116 heritage sites across the seven provinces, each with a story, audio, photos and its own 3D model. On top of this catalog it adds a heritage-yatra (tour) planner, an adaptive quiz trail with 1,160 questions, a "Find the Monument" photo game, live multiplayer heritage hunts with 696 clues, and a heritage passport that records GPS visits as stamps and badges.

1.2 Problem Statement

Visitors and learners who want to explore Nepal's heritage face the following problems:

- Reliable information on heritage sites is spread over many sources, and few are bilingual or cover all seven provinces.
- Planning a day of visits means guessing distances and opening hours; no tool combines them with walking and taxi times.
- Photo lists do not convey the shape of a temple; sites have no 3D model a visitor can examine.
- Fixed quizzes are too easy for some learners and too hard for others, and missed questions are not revisited at a useful time.

- Nothing rewards actually visiting a site, and learning is solitary: there is little a class or group of friends can play together.

1.3 Objectives

1. To build a bilingual (English and Nepali) catalog of 116 verified heritage sites with stories, audio guide, photo galleries and a 3D model for each site.
2. To implement a yatra planner that selects and orders sites within a time budget, using walking and taxi legs, and to compare its result with an exhaustive search on small inputs.
3. To implement an adaptive quiz with Elo ratings for players and questions and SM-2 spaced repetition for missed questions.
4. To implement a map-based "Find the Monument" game scored by haversine distance, and live multiplayer hunts over Socket.io with server-timed rounds and a live leaderboard.
5. To provide a heritage passport with GPS verified stamps, badges and a leaderboard, and a recommender that explains its suggestions.
6. To provide secure accounts (email verification, hashed passwords, token sessions), an admin area for content, and server-side answers and scores.

1.4 Scope and Limitations

Scope. The system is a web application used from desktop and phone browsers. It covers the features listed in the objectives.

Limitations. The yatra planner uses straight-line distance with detour factors, not live road data, so travel times are estimates. The 3D models are procedural reconstructions from researched specifications, not laser scans. Opening hours come from public sources and may change. Live hunts need an internet connection. There is no native mobile app or offline mode.

1.5 Significance of the Study

For visitors and students the system gathers dependable information about Nepali heritage in two languages in one place and turns it into a plan, a quiz and a game. The live hunt gives teachers a classroom activity that needs no installation. For the developer it combines

several real algorithms (orienteering, 2-opt, Elo, SM-2, haversine), real-time communication and 3D graphics in one product.

CHAPTER 2: LITERATURE REVIEW

Serious games for cultural heritage have been surveyed by Anderson et al. [2], who conclude that games can teach heritage content effectively when the game mechanic is tied to the real place and its story. Hamro Prachin Sampada follows this advice: every game round (a clue, a photo, a stamp) is drawn from a specific site in the catalog.

Choosing which sites to show a user is a recommendation problem. Adomavicius and Tuzhilin [3] describe content-based, collaborative and hybrid approaches and the problem of recommending items that are too similar to each other. This project uses a content-based score from tag similarity, combined with a Bayesian rating, a novelty term and nearness, followed by a diversity step.

The adaptive quiz is based on the Elo rating system [4], originally for chess, in which a player and an opponent are both rated and each result moves both ratings. Treating a question as the opponent gives a difficulty estimate that improves with use. Missed questions are scheduled for review using the SM-2 algorithm, which belongs to the family of spacing methods that grew out of the work of Wozniak and Gorzelanczyk on optimizing repetition intervals [5].

A day trip with a time limit is an instance of the orienteering problem: choose and order a subset of locations to maximize collected value within a budget. Vansteenwegen et al. [6] survey the problem and its time-window variants. The 2-opt move, first described by Croes [7], reverses a segment of a route when that shortens it. These are the methods used in the planner. Distances between map points are computed with the haversine formula [8].

Live multiplayer play needs a persistent two-way channel between browser and server. The WebSocket protocol [9] provides this, and the Socket.io library is built on it. Account sessions use JSON Web Tokens [10]. The existing literature treats these topics separately; this project combines them in a single heritage product for Nepal, and no comparable bilingual system covering all seven provinces with a 3D model of every site was found during the background study.

CHAPTER 3: METHODOLOGY

3.1 Requirement Identification

3.1.1 Study of Existing System

At present a person who wants to learn about a heritage site reads an encyclopedia entry or a travel blog, looks at the location on a general map application and, if curious, searches for photographs. Quizzes, if any, are fixed lists in printed books or social media posts. There is no connected system, which led to the requirements below, together with the CACS452 project guidelines [12].

3.1.2 Functional Requirements

Table 3.1: Main functional requirements

No.	Requirement	Actor
1	Register with email code, sign in, reset password	Visitor
2	Browse, search and filter sites on a map and list; view story, gallery and 3D model	Visitor
3	Plan a yatra within a time budget and share it by link	Visitor
4	Play the adaptive quiz, with review of missed questions	Explorer
5	Play Find the Monument (valley or all-Nepal scope)	Explorer
6	Create or join a live hunt with a 6-letter code and play timed rounds	Explorer
7	Collect GPS stamps and badges in the passport; rate and favorite sites	Explorer
8	View the global leaderboard and personal recommendations	Explorer
9	Manage sites, questions, clues and users	Administrator
10	Send verification and reset emails	System

3.1.3 Non-functional Requirements

- **Security:** hashed passwords, token sessions, role checks on admin routes, input cleaning, answers checked only on the server.
- **Performance:** common pages within about one second on a laptop; live hunt updates delivered within a second.
- **Usability:** bilingual interface, responsive layout from phone to desktop, light, dark and outdoor high-contrast themes.
- **Accessibility:** colors checked for WCAG contrast; keyboard operation of the main controls.

- **Portability:** runs on any system with Node.js and MySQL; the repository includes scripts for a portable local MySQL.

3.1.4 Feasibility Study

Technical feasibility. React, Vite, Express, Socket.io, three.js, MapLibre GL and MySQL are open source and run on an ordinary laptop. The developer has worked with JavaScript and relational databases. **Operational feasibility.** A visitor needs only a browser; no installation is required. The admin area uses plain forms. **Economic feasibility.** There are no licence fees: map tiles come from OpenFreeMap, photographs from Wikimedia Commons, textures from Poly Haven (CC0) and fonts from Google Fonts. Hosting for a demonstration can use a small virtual server. **Schedule feasibility.** The work is planned over seventeen weeks (Chapter 4).

3.2 Development Model

An incremental and iterative model [11] will be followed. Each cycle adds one slice (for example the catalog, then the planner, then the quiz), tests it and corrects defects before the next slice begins. This suits a single developer who must keep the data model, API and interface consistent, and it gives a working demonstration after every cycle.

3.3 System Design

3.3.1 System Architecture

The system has three tiers. A React single page client renders the interface, the MapLibre map and the three.js scenes. An Express server exposes about fifty REST endpoints and hosts the Socket.io server for live hunts. A MySQL database stores users, sites, questions, clues and progress. Verification and reset codes are sent by email through Gmail SMTP using Nodemailer; when no mail account is set, messages are saved to an outbox folder for preview.

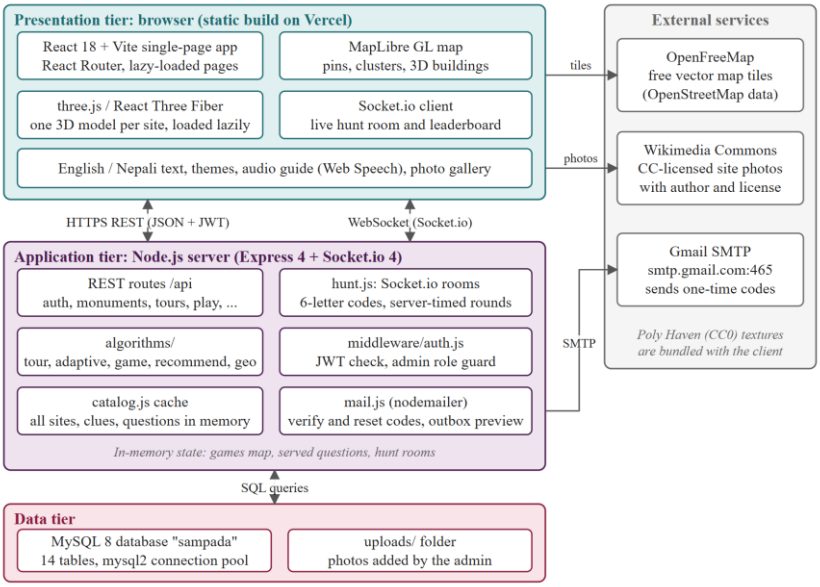


Figure 3.1: Proposed system architecture

3.3.2 Use Case Diagram

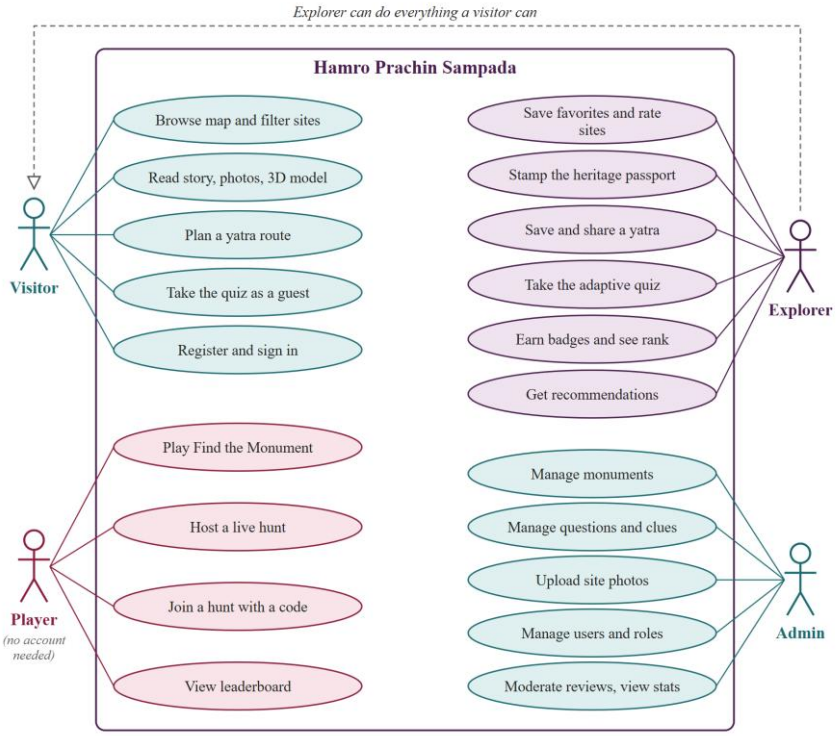


Figure 3.2: Use case diagram

Table 3.2: Use case: Play a live heritage hunt

Item	Description
Actor	Explorer (host and players)
Precondition	All players are signed in; the host has chosen the number of rounds and the scope

Item	Description
Main flow	1. The host creates a room and receives a 6-letter code. 2. Players join with the code and see the lobby. 3. The host starts; after a 3-2-1 countdown the server sends the first clue. 4. Each player places a guess on the map; the server scores it by distance and time. 5. After all answers or the time limit, the server reveals the site and updates the leaderboard. 6. Final standings are saved to the global leaderboard.
Postcondition	Scores are stored for each player; badges are awarded where earned

3.3.3 Context Diagram and Data Flow

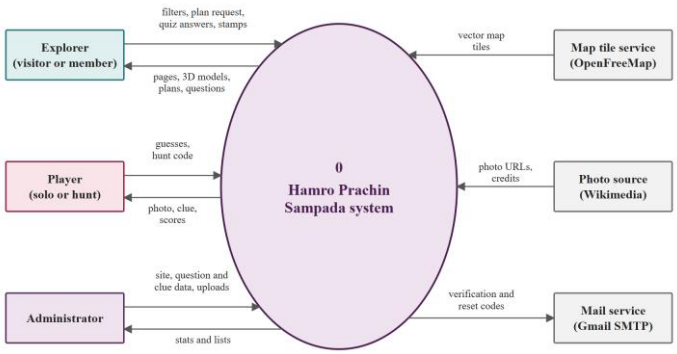


Figure 3.3: Context diagram (DFD level 0)

Level 1 splits the system into catalog browsing, yatra planning, quiz and review, photo game and live hunt, passport and badges, and administration. The planning, quiz and game processes read from the catalog store and write scores and progress to the user store.

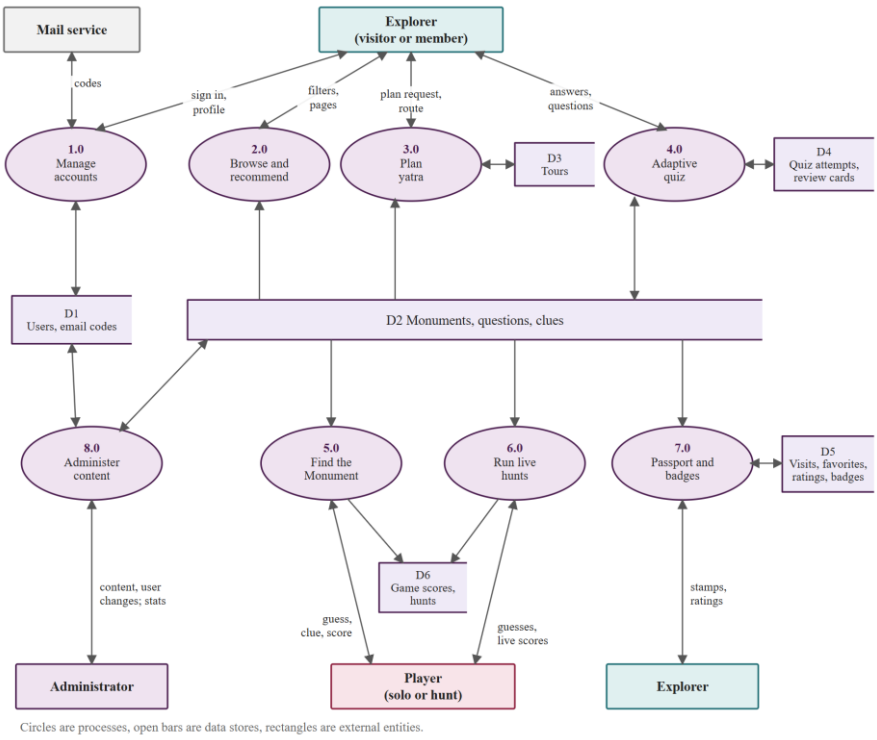
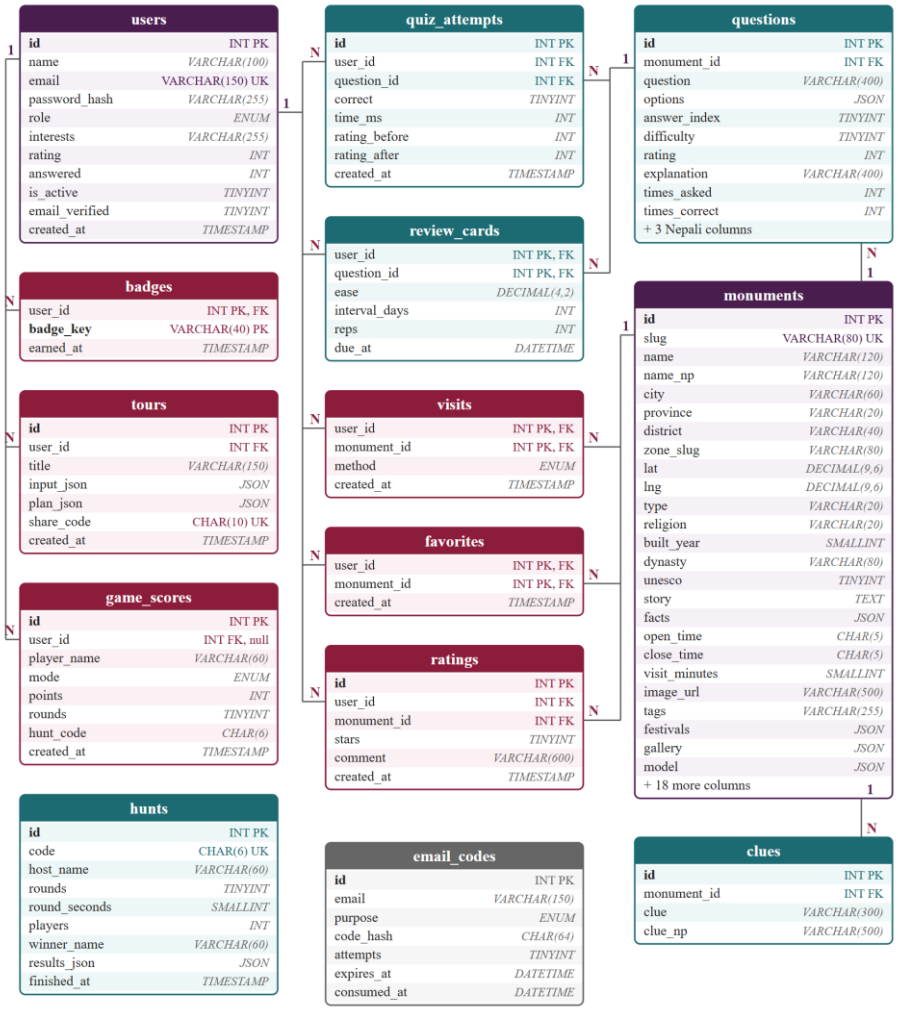


Figure 3.4: DFD level 1

3.3.4 Database Design

The database has fourteen tables: users, email_codes, monuments, questions, clues, quiz_attempts, review_cards, visits, favorites, ratings, tours, game_scores, hunts and badges. A `monuments` row holds names in both languages, coordinates, province, tags, story text, opening hours, fee and the model specification. `review_cards` stores the SM-2 state (ease, interval, repetitions) of each missed question per user, and `visits` stores GPS stamps. Foreign keys cascade on user deletion.



PK primary key, FK foreign key, UK unique key. 1 and N mark the one and many sides of each relationship.
email_codes is tied to users by email address, and hunts to game_scores by hunt_code, without foreign keys.

Figure 3.5: Entity relationship diagram

3.3.5 Algorithms in Brief

Table 3.3: Algorithms to be implemented

Algorithm	Used for	Notes
Orienteering heuristic	Yatra planner: choose and order sites in a time budget	Greedy insertion by value per minute, 2-opt, re-insertion; checked against brute force
Haversine distance	Leg lengths, game scoring, GPS stamp check	O(1) per pair
Elo rating	Player and question ratings	Next question chosen at about 70% predicted success
SM-2	Review schedule of missed questions	Ease factor and growing interval
Distance score	Find the Monument points	Exponential decay with distance plus a speed bonus; maximum 1,000

Algorithm	Used for	Notes
Tag cosine and Bayesian rating	Recommender	Plus novelty, nearness and city diversity re-ranking
Room state machine	Live hunt	Lobby, countdown, round, reveal, final; grace time for reconnecting
bcrypt, JWT	Passwords and sessions	Library primitives

In the planner each site has an estimated visit time and an opening window. Starting from the traveler's location, the heuristic repeatedly inserts the site with the best value per added minute where the opening window and time budget still hold, then applies 2-opt to shorten the route and tries to re-insert skipped sites. A leg longer than 2.5 km is assumed to be taken by taxi, otherwise on foot.

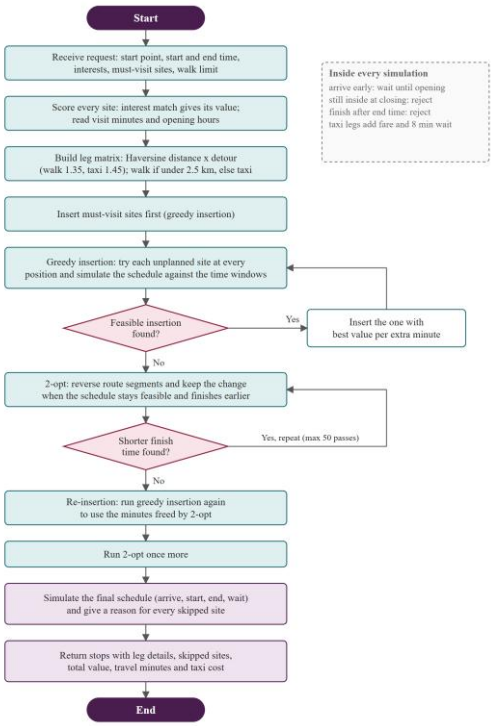


Figure 3.6: Flow of the yatra planner

3.3.6 Interface Design

The interface will use one design system with four palette pairs (plum, wine red, persian and lilac), Cormorant Garamond and Jost typography for English and Martel and Hind for Devanagari, temple-arch photo frames and light, dark and outdoor themes. The planned pages are listed below.

Table 3.4: Planned pages

Area	Pages
Public	Home, explore (map and list), monument detail with 3D model, walk planner, shared walk, about, credits, sign in, register, reset password
Explorer	Quiz trail, Find the Monument, live hunt, passport, leaderboard
Administrator	Admin panel for sites, questions, clues and users

3.4 Tools

Table 3.5: Proposed tools

Area	Tools
Frontend	React 18, Vite, React Router, Framer Motion, MapLibre GL, three.js with React Three Fiber, Socket.io client
Backend	Node.js, Express, Socket.io, mysql2, bcryptjs, jsonwebtoken, Nodemailer
Database	MySQL 8.4 (portable copy inside the project, port 3307)
Content	Wikidata and OpenStreetMap coordinates, Wikimedia Commons photos, Poly Haven textures
Version control and testing	Git and GitHub; node:test; API flow script; hunt simulator

3.5 Testing Plan

Unit tests will cover the algorithms: the planner against brute force on small instances, Elo updates, SM-2 intervals, distance scoring in both scopes, and the recommender. API tests will run complete flows (register, verify, quiz, game, passport, admin permissions) against a freshly seeded database. A simulator script will run a three-player live hunt over Socket.io, including a disconnect and reconnect. Manual browser checks will cover desktop and phone widths and the three themes.

Table 3.6: Test plan

Level	What is checked	Tool
Unit	Tour planner vs brute force, Elo, SM-2, scoring, recommender	node:test
API flows	Authentication, quiz, games, passport, admin permissions, validation	Node script
Real-time	Three-player hunt, round timing, leaderboard order, reconnect	Hunt simulator
Interface	Pages, forms, map, 3D viewer, responsive widths, themes	Manual in browser

CHAPTER 4: GANTT CHART

The work is planned over seventeen weeks as shown in Figure 4.1: requirement study and proposal (weeks 1 to 2), site data and photos (1 to 6), database and API design (3 to 5), algorithms (5 to 9), frontend (6 to 12), 3D models (8 to 13), live hunts and passport (11 to 14), testing (12 to 16), and documentation and defense (14 to 17). Data collection starts first because every feature depends on the catalog. Interface work overlaps with the algorithms, and testing runs alongside the later implementation.

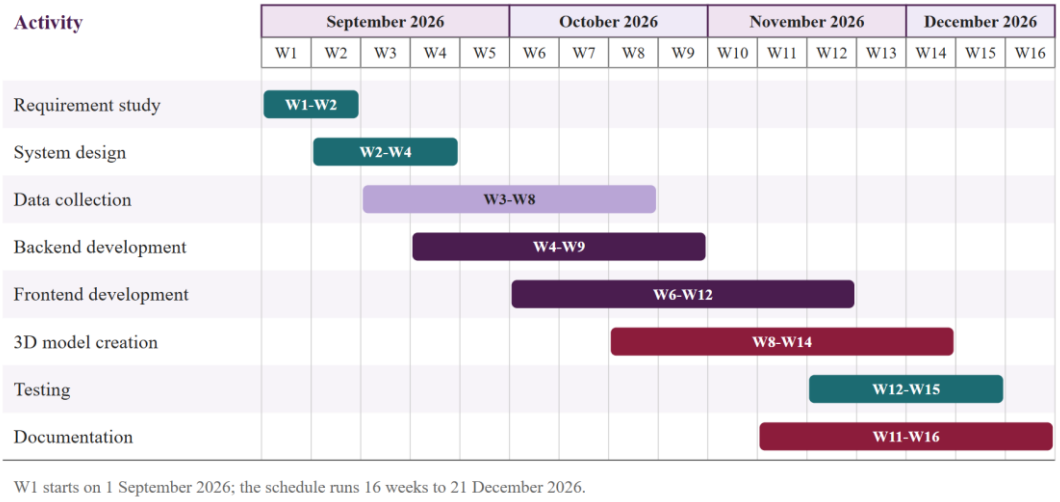


Figure 4.1: Project schedule

CHAPTER 5: EXPECTED OUTCOME

At the end of the project the following outcomes are expected:

- A working bilingual web application with a catalog of 116 heritage sites across the seven provinces, each with a story, audio, gallery and 3D model.
- A yatra planner whose routes stay within the chosen time budget and match brute force on small inputs.
- An adaptive quiz of 1,160 questions that adjusts to the player and schedules missed questions for review.
- A Find the Monument game and live multiplayer hunts (696 clues) whose scoring is done on the server.
- A heritage passport with GPS stamps, badges, a leaderboard and explained recommendations.
- Automated tests (unit, API and hunt simulation), a project report, this proposal and a presentation.

REFERENCES

[1] UNESCO World Heritage Centre, "Kathmandu Valley." [Online]. Available: https://whc.unesco.org/en/list/121. [Accessed: 4 Nov. 2026].

[2] E. F. Anderson, L. McLoughlin, F. Liarokapis, C. Peters, P. Petridis, and S. de Freitas, "Serious games in cultural heritage," in *Proc. 10th Int. Symp. Virtual Reality, Archaeology and Cultural Heritage (VAST), State of the Art Reports*, Valletta, Malta, Sep. 2009, pp. 29-48.

[3] G. Adomavicius and A. Tuzhilin, "Toward the next generation of recommender systems: A survey of the state-of-the-art and possible extensions," *IEEE Trans. Knowl. Data Eng.*, vol. 17, no. 6, pp. 734-749, Jun. 2005.

[4] A. E. Elo, *The Rating of Chessplayers, Past and Present*. New York, NY, USA: Arco, 1978.

[5] P. A. Wozniak and E. J. Gorzelanczyk, "Optimization of repetition spacing in the practice of learning," *Acta Neurobiol. Exp.*, vol. 54, no. 1, pp. 59-62, 1994.

[6] P. Vansteenwegen, W. Souffriau, and D. Van Oudheusden, "The orienteering problem: A survey," *Eur. J. Oper. Res.*, vol. 209, no. 1, pp. 1-10, Feb. 2011.

[7] G. A. Croes, "A method for solving traveling-salesman problems," *Oper. Res.*, vol. 6, no. 6, pp. 791-812, Dec. 1958.

[8] R. W. Sinnott, "Virtues of the haversine," *Sky and Telescope*, vol. 68, no. 2, p. 159, 1984.

[9] I. Fette and A. Melnikov, "The WebSocket protocol," RFC 6455, IETF, Dec. 2011.

[10] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, IETF, May 2015.

[11] I. Sommerville, *Software Engineering*, 10th ed. Harlow, U.K.: Pearson, 2016.

[12] Tribhuvan University, FOHSS, "CACS452 Project III, BCA eighth semester course outline." [Online]. Available: https://www.itcollegenepal.com/wp-content/uploads/2022/02/CACS452-BCA-VIII-SEM-Project-III.pdf. [Accessed: 2 Nov. 2026].